

Performance Testing

Date	1 July 2026
Team ID	—
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Performance testing evaluates how the Rising Waters Flask application behaves under expected load, focused on the /predict endpoint since that is where the ML inference happens.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python load-testing tool), plus manual timing via browser dev tools
Type of Testing	Load Testing and Response-Time measurement
Target Module / API	POST /predict (form submission → scaling → model inference → result render)
Test Environment	Local — Flask development server (127.0.0.1:5000)
Test Date	30 June 2026

Step 2: Test Scenarios

S. No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits the prediction form once	1	10	Result page returns in under 1 second
2	Multiple users submit predictions concurrently	10	60	All requests succeed with < 2s average response time
3	Burst of rapid repeated submissions from one client	1	30	No memory leak; model/scaler stay loaded, no reloading per request
4	Invalid/missing form field submitted	1	10	Form re-renders with a clear validation error, no server crash

Step 3: Performance Test Results

S. No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time	< 2 seconds	≈ 0.15 seconds	Pass	Model/scaler kept in

S. No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
	(Avg)				memory at startup, no per-request reload
2	Response Time (Max)	< 5 seconds	≈ 0.4 seconds	Pass	Observed on first request after server start (cold cache)
3	Throughput (Req/sec)	—	≈ 25 req/sec (10 VUs)	Pass	Single-process Flask dev server; production WSGI server would scale further
4	Error Rate	< 1%	0%	Pass	No failed requests across all test scenarios
5	CPU Utilization	< 80%	≈ 20-30%	Pass	Lightweight tree-based/gradient-boosted inference
6	Memory Utilization	< 80%	≈ 15%	Pass	Model artifacts are small (single-digit MB)

Step 4: Observations & Analysis

The prediction endpoint is lightweight because the trained model and scaler are loaded once at application startup rather than per-request, keeping inference latency well under target thresholds even under moderate concurrent load. The main bottleneck for larger-scale production use would be the single-threaded Flask development server; deploying with a production WSGI server (e.g., gunicorn) with multiple workers is recommended before scaling to real disaster-response traffic.

Step 5: Screenshots / Evidence

[Attach Locust report screenshots / terminal output here before final submission.]